CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

We set out to check how well Llama-2-7B-Chat holds up when the same harmful intent is asked in different ways. Using 2,500 base questions × 3 variants (7,500 total) and a small ensemble of safety detectors, we saw something simple but important:

- Direct harmful prompts are mostly blocked (ASR ≈ 5.6%, defense ≈ 94.4%).
- A light rewording already hurts robustness significantly (Simple paraphrase ASR ≈ 39.0%).
- Carefully engineered jailbreaks flip the outcome (Adversarial ASR ≈ 56.2%).

Looking at each base question pairwise, about 56% of the ones that looked safe in the direct form turn unsafe under the adversarial variant (True ASR ≈ 56%). Some policy areas are clearly weaker (Illegal Activity, Physical Harm, Malware), and the detectors agree less often on adversarial cases—useful as a small "be careful" signal. The whole pipeline ran on a single GPU with 4-bit quantization, so the setup is practical even for a college lab.

One-line summary: baseline alignment is reasonable for obvious prompts, but it doesn't generalize to meaning-preserving paraphrases or jailbreak styles. If we want safer systems, we have to handle intent, not just keywords.

5.2 Future Work (Semantic Smoothing Defense Implementation)

What We've Completed (Steps 1-3)

Following the workflow diagram:

1. **Data Collection / Source Set** ☑
   - Curated 2,500 safety-sensitive prompts covering 14 content policy categories
   - Built three-variant dataset: Direct, Simple Paraphrase, Adversarial Paraphrase
2. **Prompt Variants Generation** ☑
   - 2A: Original raw prompts
   - 2B: Simple paraphrases (semantic-preserving, temp=0.7 sampling)
   - 2C: Adversarial paraphrases (detector-guided jailbreak templates)
3. **Baseline Evaluation (Llama-2 Model)** ☑
   - Recorded metrics: ASR, defense rates, toxicity scores, detector agreement
   - Measured success/fooling rates across all three variants
   - Computed utility and semantic similarity baselines

Key Finding: Current ASR shows critical vulnerability (Direct: 5.6%, Simple: 39.0%, Adversarial: 56.2%)

What We Need to Do Next (Steps 4-5)

4A. Apply Semantic Smoothing Defense 🔄 (Research Gap & Next Phase)

This is the **core defense mechanism** we propose to implement:

How Semantic Smoothing Works:

- For each user prompt, generate multiple semantic paraphrases (3-5 variants)
- Run all paraphrases through Llama-2 and score each response with our ensemble detectors
- Aggregate toxicity scores across the paraphrase cluster
- **Decision rule**: If cluster average ≥ threshold (e.g., 0.5), block or ask for reformulation
- **Intuition**: Attackers craft prompts near decision boundaries; paraphrasing reveals true intent

Implementation Plan:

1. Build paraphrase generator (T5-based or GPT-style rephrasing)
2. Create cluster-level scoring pipeline
3. Tune aggregation strategy (mean, max, majority vote)
4. Calibrate threshold to balance safety vs. false positives
5. Measure latency overhead (target: <2× baseline inference time)

Research Gap We Address:

- Current defenses check single prompts in isolation (brittle to surface variations)
- Semantic Smoothing checks *intent robustness* across paraphrase neighborhoods
- Fills gap between pattern-matching filters and computationally expensive gradient-based defenses

Expected Outcome:

- Reduce ASR on adversarial prompts from 56.2% to <20%
- Maintain utility: false positive rate <5% on benign queries

4B. Upgrade Defense Test (Comparative Validation)

Once Semantic Smoothing is implemented:

- Test on same 7,500-case dataset with upgraded defense active
- Record new ASR, fooling rates, semantic similarity checks
- Compare to baseline: ΔASR, latency cost, false positive rate

Alternatives to Compare:

- Ensemble detector upgrades (tuning transforms, adversarial training)
- Uncertainty-aware gating (reject when detectors disagree)
- Combined: Semantic Smoothing + ensemble uncertainty

5. Analyze & Report (Final Phase)

Produce comprehensive comparison:

- Success/Fooling Rate reduction (how much safer?)
- Robustness vs. Utility tradeoffs (are we over-blocking?)
- Semantic similarity checks (does defense preserve user intent?)

- Statistical significance tests (is improvement real?)
- Deployment readiness assessment (latency, scalability, failure modes)

Deliverable

Phase-2 Technical Report:

- Implementation: Semantic Smoothing pipeline (code + config)
- Results: ASR reduction plots, policy-wise gains, paired before/after analysis
- Trade-offs: Latency overhead, false positive analysis, utility benchmarks
- Deployment guide: Threshold tuning, integration with existing safety layers

Research Contribution: First college-lab-scale demonstration that intent-level defenses (paraphrase clustering) can substantially reduce jailbreak success rates without expensive retraining or gradient-based optimization.